COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	July 09, 2026	**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create Code That Follows the Basic C++ Code Structure

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then
- return true if it was a success.

Note:

- The driver program must call each function.
- The definitions must be after the main function.

Sections	Answer
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	

Class Declaration and Method Definition Section	`class NumberOperations {`
	`public:`
	`  int sumofTwoNumbers(int fNum, int lNum);`
	`  bool isGreater(int A, int B);`
	`  bool logicalOperations(bool a, bool b);`
	`};`

Main Function	`int main() {`
	`  NumberOperations ops;`
	`  int num1, num2;`
	
	`  std::cout << "Enter two numbers: ";`
	`  std::cin >> num1 >> num2;`
	`  std::cout << "Sum: " <<`
	`ops.sumofTwoNumbers(num1, num2) << std::endl;`

```
    std::cout << std::boolalpha;

    std::cout << "\nIs " << num1 << " greater
than " << num2 << "? "
              << ops.isGreater(num1, num2) <<
std::endl;

    bool boolA, boolB;
    std::cout << "\nEnter two boolean values (1
or 0): ";
    std::cin >> boolA >> boolB;

    bool success = ops.logicalOperations(boolA,
boolB);
    std::cout << "Logical operations completed
successfully: " << success << std::endl;

    return 0;
}
```

Method Definition

```
int NumberOperations::sumofTwoNumbers(int fNum,
int lNum) {
    int total = fNum + lNum;
    return total;
}

bool NumberOperations::isGreater(int A, int B) {
    return A > B;
}

bool NumberOperations::logicalOperations(bool a,
bool b) {
    std::cout << std::boolalpha;
    std::cout << "\n Results: x`\n";
    std::cout << "A = " << a << ", B = " << b <<
std::endl;

    std::cout << "A AND B: " << (a && b) <<
std::endl;
```

```
    std::cout << "A OR B: " << (a || b) <<
std::endl;
    std::cout << "NOT A: " << (!a) << std::endl;
    std::cout << "NOT B: " << (!b) << std::endl;

    return true;
}
```

Table 1-1. C++ Structure Code for Answer (ILO A)

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class triangle will be implemented.

```
#include <iostream>

class Triangle{
    private:
    double totalAngle, angleA, angleB, angleC;
    public:
        //constructor
        Triangle(double A, double B, double C);

        //methods
        void setAngles(double A, double B, double C);
```

```
        //validation
        const bool validateTriangle();
};

int main() {

    //drive code
    Triangle set1(40,30,210);
    if(set1.validateTriangle()) {
        std::cout << "The shape is a valid Triangle" << std::endl;
    } else {
        std::cout << "The shape is NOT a valid Triangle. \n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Observations:

- Logical and Mathematical Issues
- After removing the compilation errors, there are some issues with logic about the geometry. For validating, the method checks if the sum of all angles is less than or equal to 180 degrees.

However, according to the Euclidean geometry rules, the angles of any triangle should be summing up to 180 degrees exactly. Moreover, each angle should be greater than zero degrees. According to the logic of the current program, a combination of angles such as 10, 20 and 30 degrees (the sum equals 60) is regarded as a valid triangle. ● Behavior of Expected Output - As soon as we resolve the compiler error, the run of the program with test case angles 40, 30 and 210 degrees gives us a sum of 280 degrees. As 280 degrees are not less than or equal to 180 degrees, then the method validateTriange returns false. That means that the else block of the main method is performed by the program, and we get the output message, "The shape is NOT a valid Triangle."

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language
The supplementary activities are meant to gauge your ability in using C++. The problems below range from easy to intermediate to advanced problems. Note your difficulties after answering the problems below.

1. Create a C++ program to swap the two numbers in different variables.

```
#include <iostream>

int main() {
    double num1, num2, temp;

    std::cout << "Enter first number: ";
    std::cin >> num1;
    std::cout << "Enter second number: ";
    std::cin >> num2;

    std::cout << "\nBefore swapping:\n";
    std::cout << "num1 = " << num1 << ", num2 = " << num2 << std::endl;

    temp = num1;
    num1 = num2;
    num2 = temp;

    std::cout << "\nAfter swapping:\n";
```

```
    std::cout << "num1 = " << num1 << ", num2 = " << num2 << std::endl;

    return 0;
}
```

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```
#include <iostream>

double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9.0 / 5.0 + 32;
}

int main() {
    double kelvin;

    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;

    double fahrenheit = kelvinToFahrenheit(kelvin);

    std::cout << kelvin << " K is equal to " << fahrenheit << " F" <<
std::endl;

    return 0;
}
```

3. Create a C++ program that has a function that will calculate the distance between two points.

```
#include <iostream>
#include <cmath>

double calculateDistance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}

int main() {
    double x1, y1, x2, y2;

    std::cout << "Enter coordinates of point 1 (x1 y1): ";
    std::cin >> x1 >> y1;
    std::cout << "Enter coordinates of point 2 (x2 y2): ";
    std::cin >> x2 >> y2;
```

```
    double distance = calculateDistance(x1, y1, x2, y2);

    std::cout << "The distance between the two points is: " << distance
<< std::endl;

    return 0;
}
```

4. Modify the code given in ILO B and add the following functions:
    a) A function to compute for the area of a triangle
    b) A function to compute for the perimeter of a triangle
    c) A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
#include <iostream>
#include <cmath>
#include <string>

class Triangle {
private:
    double sideA, sideB, sideC;

public:
    Triangle(double a, double b, double c);

    void setSides(double a, double b, double c);

    bool validateTriangle() const;

    double getArea() const;

    double getPerimeter() const;

    std::string classifyTriangle() const;
};

int main() {
    Triangle set1(3, 4, 5);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid Triangle.\n";
        std::cout << "Perimeter: " << set1.getPerimeter() << "\n";
        std::cout << "Area: " << set1.getArea() << "\n";
        std::cout << "Type: " << set1.classifyTriangle() << "\n";
```

```
    } else {
        std::cout << "The shape is NOT a valid Triangle. \n";
    }

    std::cout << "\n-----------------------------\n\n";

    Triangle set2(10, 2, 3);
    if (set2.validateTriangle()) {
        std::cout << "The shape is a valid Triangle.\n";
    } else {
        std::cout << "The shape is NOT a valid Triangle. \n";
    }

    return 0;
}

Triangle::Triangle(double a, double b, double c) {
    sideA = a;
    sideB = b;
    sideC = c;
}

void Triangle::setSides(double a, double b, double c) {
    sideA = a;
    sideB = b;
    sideC = c;
}

bool Triangle::validateTriangle() const {
    return (sideA + sideB > sideC) &&
           (sideA + sideC > sideB) &&
           (sideB + sideC > sideA) &&
           (sideA > 0 && sideB > 0 && sideC > 0);
}

double Triangle::getArea() const {
    if (!validateTriangle()) return 0.0;
    double s = getPerimeter() / 2.0;
    return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

double Triangle::getPerimeter() const {
```

```
    if (!validateTriangle()) return 0.0;
    return sideA + sideB + sideC;
}

std::string Triangle::classifyTriangle() const {
    if (!validateTriangle()) {
        return "Invalid Triangle";
    }

    double a = sideA;
    double b = sideB;
    double c = sideC;

    if (a >= b && a >= c) {
        c = sideA;
        a = sideB;
        b = sideC;
    } else if (b >= a && b >= c) {
        c = sideB;
        a = sideA;
        b = sideC;
    }

    double sumOfSquares = (a * a) + (b * b);
    double longestSquare = c * c;

    if (sumOfSquares > longestSquare) {
        return "Acute-angled";
    } else if (sumOfSquares < longestSquare) {
        return "Obtuse-angled";
    } else {
        return "Others (Right-angled)";
    }
}
```

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance (error check)**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Whenever an error pops up, I ask Gemini what's causing the error.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.